Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Jijel
Faculté des Sciences Exactes et Informatique
Département d'Informatique

Master 1 — Intelligence Artificielle
Module : Recherche Intelligente et Collaborative

Cartographie d'une Fonction Mystère

Implémentation d'un Perceptron Multicouche (MLP) avec NumPy

Année Universitaire 2025 / 2026

Table des matières

1 Introduction **2**

2 Génération du Dataset **2**
2.1 Points d'entraînement 2
2.2 Normalisation 2
2.3 Visualisation de la vérité terrain 2

3 Architecture du Réseau **2**
3.1 Classe MLP 3
3.2 Initialisation des poids 3
3.3 Fonctions d'activation 3
3.4 Fonction de perte 3

4 Implémentation de la Rétropropagation **3**
4.1 Propagation avant (forward pass) 3
4.2 Rétropropagation (backward pass) 4

5 Optimisation et Hyperparamètres **4**
5.1 Algorithme : SGD avec Momentum 4
5.2 Tableau des hyperparamètres 4

6 Résultats et Visualisations **5**
6.1 Courbe d'apprentissage 5
6.2 Comparaison MLP vs vérité terrain 5

7 Questions de Réflexion **5**
7.1 Impact de l'activation : ReLU vs Linéaire 5
7.2 Capacité du réseau : architecture réduite [2, 4, 1] 5
7.3 Effet du Momentum sur la convergence 5

8 Conclusion **6**

1. Introduction

Ce rapport présente la réalisation complète du TP portant sur la **cartographie d'une fonction mathématique complexe** à l'aide d'un Perceptron Multicouche (MLP). L'objectif est d'implémenter ce réseau *from scratch*, en utilisant uniquement la bibliothèque `NumPy`, sans recourir à des frameworks de haut niveau comme PyTorch ou TensorFlow.

Le réseau doit apprendre à approximer la surface suivante :

$$f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5 \cdot \cos(2x + 2y)$$

Cette fonction combine des oscillations **radiales** $\left(\sin(\sqrt{x^2 + y^2})\right)$ et **angulaires** $\left(0.5\cos(2x+2y)\right)$, ce qui en fait un excellent banc d'essai pour évaluer la capacité d'approximation non linéaire d'un MLP.

2. Génération du Dataset

2.1. Points d'entraînement

Conformément aux consignes, **2 000 points** (x, y) ont été générés aléatoirement avec une distribution uniforme sur le domaine $[-5, 5]^2$. La graine aléatoire a été fixée à 42 (`np.random.seed(42)`) pour garantir la reproductibilité des expériences.

2.2. Normalisation

Une étape de **standardisation (z-score)** a été appliquée aux entrées **et** aux sorties afin de faciliter la convergence du réseau :

$$X_{\text{norm}} = \frac{X - \mu_X}{\sigma_X}, \qquad z_{\text{norm}} = \frac{z - \mu_z}{\sigma_z}$$

Les paramètres (μ, σ) calculés sur l'ensemble d'entraînement sont conservés pour dénormaliser les prédictions lors de la phase d'évaluation.

2.3. Visualisation de la vérité terrain

La vérité terrain a été affichée à l'aide d'un *scatter plot* 3D (projection `'3d'` de `matplotlib`), en colorant les 2 000 points selon la valeur de z avec la colormap `viridis`. Ce graphique met en évidence la complexité de la surface à apprendre.

3. Architecture du Réseau

3.1. Classe MLP

Le MLP a été implémenté sous forme d'une **classe Python flexible** acceptant une liste de tailles de couches (`layer_sizes`). L'architecture choisie est :

Couche	Entrée	Sortie	Activation	Paramètres
Couche 1 (cachée)	2	64	ReLU	192
Couche 2 (cachée)	64	64	ReLU	4 160
Couche 3 (sortie)	64	1	Linéaire	65
Total				**4 417**

3.2. Initialisation des poids

L'**initialisation de He** a été utilisée, adaptée aux réseaux ReLU, pour éviter la disparition du gradient :

$$\sigma_{\text{init}} = \sqrt{\frac{2}{\text{fan_in}}}$$

Les biais sont initialisés à zéro.

3.3. Fonctions d'activation

- **ReLU** pour les couches cachées : $\text{ReLU}(z) = \max(0, z)$, introduit la non-linéarité indispensable pour apprendre des fonctions complexes.
- **Linéaire (identité)** pour la couche de sortie, adaptée à la régression.

3.4. Fonction de perte

L'Erreur Quadratique Moyenne (MSE) a été implémentée :

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{z}_i - z_i\right)^2$$

4. Implémentation de la Rétropropagation

4.1. Propagation avant (forward pass)

La méthode `forward()` parcourt chaque couche, calcule les pré-activations $Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$, applique l'activation, et stocke tous les états intermédiaires (Z, A) nécessaires à la rétropropagation.

Tout le calcul est **vectorisé** via les opérateurs matriciels de NumPy, traitant des mini-batches entiers en une seule opération.

4.2. Rétropropagation (backward pass)

La méthode `backward()` implémente manuellement la **règle de la chaîne**. Le calcul remonte de la dernière couche vers l'entrée :

1. **Gradient initial (sortie)** :

$$\delta = \frac{2}{m}\big(A^{(\text{out})} - y\big)$$

2. **Gradients des poids et biais** pour chaque couche i :

$$\nabla W^{(i)} = A^{(i)\top}\delta, \qquad \nabla b^{(i)} = \sum \delta$$

3. **Propagation vers la couche précédente** :

$$\delta \leftarrow \big(\delta\, W^{(i)\top}\big) \odot \mathrm{ReLU}'\big(Z^{(i-1)}\big)$$

Conformément à la contrainte du TP, **aucune boucle for n'itère sur les exemples** : tout est vectorisé matriciellement.

5. Optimisation et Hyperparamètres

5.1. Algorithme : SGD avec Momentum

L'optimisation utilise une descente de gradient stochastique enrichie d'un terme de **Momentum** ($\beta = 0.9$) :

$$v_W \leftarrow \beta\, v_W - \eta\, \nabla W, \qquad W \leftarrow W + v_W$$

Le Momentum accélère la convergence en accumulant de l'élan dans les directions persistantes et atténue les oscillations dues au bruit du mini-batch.

5.2. Tableau des hyperparamètres

Hyperparamètre	Valeur	Justification
Learning rate η	5×10^{-3}	Compromis vitesse/stabilité
Momentum β	0.9	Valeur classique, réduit les oscillations
Taille de batch	128	Équilibre bruit du gradient et vitesse
Nombre d'époques	800	Dans la fourchette 500–1000, convergence atteinte
Split train/val	80/20%	Surveiller le sur-apprentissage

6. Résultats et Visualisations

6.1. Courbe d'apprentissage

La courbe de loss (MSE en fonction des époques) montre une **descente régulière** lors des premières époques, suivie d'un plateau de convergence. L'écart entre les courbes train (bleu) et validation (rouge) reste faible, indiquant l'absence de sur-apprentissage.

6.2. Comparaison MLP vs vérité terrain

Après entraînement, une grille régulière 100×100 a été générée sur $[-5, 5]^2$. Les prédictions ont été dénormalisées et comparées à $f(x, y)$. Trois visualisations sont affichées côte à côte :

- La **vérité terrain** $f(x, y)$ sur la grille.
- La **reconstruction** prédite par le MLP.
- La **carte d'erreur absolue** $|f - \hat{f}|$, révélant les zones difficiles (hautes fréquences).

La reconstruction est visuellement très proche de la surface originale, confirmant la capacité du MLP [2, 64, 64, 1] à approximer cette fonction complexe.

7. Questions de Réflexion

7.1. Impact de l'activation : ReLU vs Linéaire

Remplacer toutes les activations ReLU par des fonctions linéaires rendrait le réseau **équivalent à une transformation affine globale**, quelle que soit la profondeur (la composition de fonctions linéaires reste linéaire). Un tel modèle serait incapable d'apprendre $f(x, y)$, qui est hautement non-linéaire. **La non-linéarité des activations est fondamentale** pour la puissance d'approximation universelle du MLP.

7.2. Capacité du réseau : architecture réduite [2, 4, 1]

Une architecture $[2, 4, 1]$ ne dispose que de **21 paramètres**. Cette capacité est insuffisante pour capturer les oscillations de $f(x, y)$: le modèle souffrirait de *sous-apprentissage* (underfitting), avec une loss élevée et une reconstruction très lisse sans reproduire les variations radiales et angulaires.

7.3. Effet du Momentum sur la convergence

Le Momentum ($\beta = 0.9$) est déjà implémenté dans le code. Son double effet :

- **Accélère** la convergence en accumulant de l'élan dans les directions persistantes du gradient.

— **Atténue** les oscillations dues au bruit des mini-batches.

Par rapport au SGD pur, le Momentum permet typiquement d'atteindre un niveau de loss équivalent en 2 à 5 fois moins d'époques.

8. Conclusion

Ce TP a permis d'implémenter complètement un MLP en NumPy, couvrant toutes les étapes : génération et normalisation des données, architecture flexible, initialisation de He, propagation avant et rétropropagation vectorisée, optimisation par SGD avec Momentum, et évaluation visuelle.

Le code répond intégralement aux exigences du TP : architecture $[2, 64, 64, 1]$, 800 époques (dans la plage 500–1000), vectorisation stricte, courbe de loss, et comparaison sur grille. La surface apprise est visuellement fidèle à la vérité terrain, démontrant la puissance d'approximation des MLPs même implémentés *from scratch*.